

QUESTION

CASE STUDY ON CLASS & OBJECTS (CO1): Bank Account Management System design a system to manage bank accounts using object oriented programming concepts.

Classes: Account, Savings-Account, Current-Account.

Attributes: Account number, balance, account type, interest rate (for savings account), overdraft limit (for current account), etc.

Methods: Deposit, Withdraw, Calculate Interest, Transfer, etc.

Object Usage: Create objects for each customer's account and perform transactions like deposits, withdrawals, and transfers.

Steps for Execution:

1. Open any Java IDE such as IntelliJ IDEA, Eclipse, or VS Code.
2. Create a new Java project.
3. Create a package named javaprograms.
4. Inside the package, create a Java class named BankAccounts.java.
5. Copy the given code into BankAccounts.java.
6. Save the program.
7. Compile the program.
8. Run the BankAccounts class because it contains the main() method.
9. The output will be displayed in the console.

Algorithm

1. Start.
2. Create a SavingsAccount object with account number, balance, and interest rate.
3. Create a CurrentAccount object with account number, balance, and overdraft limit.
4. Display the details of the savings account.
5. Deposit money into the savings account.
6. Calculate interest for the savings account and update its balance.
7. Display the details of the current account.
8. Withdraw money from the current account using the overdraft limit.

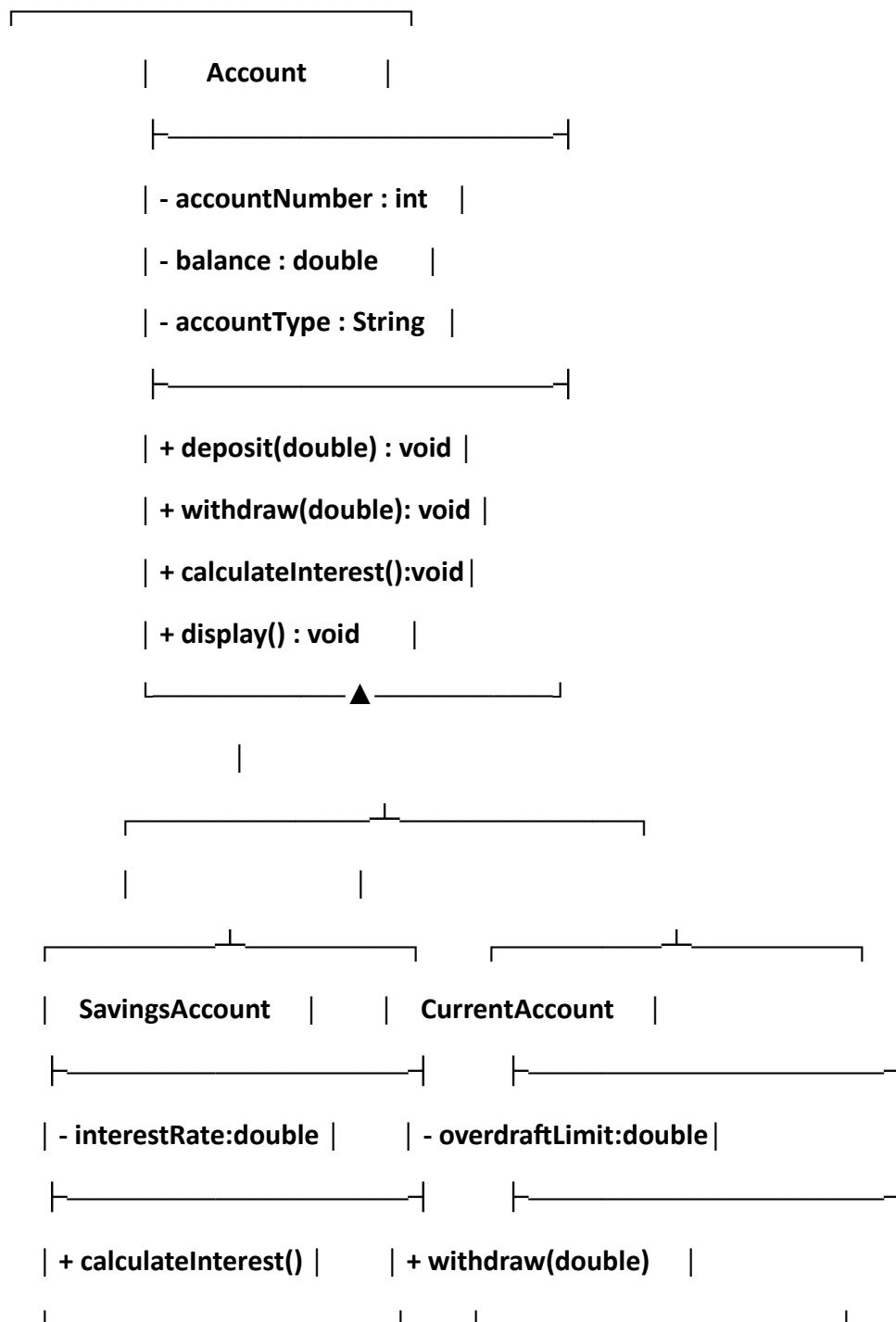
9.Transfer money from the savings account to the current account.

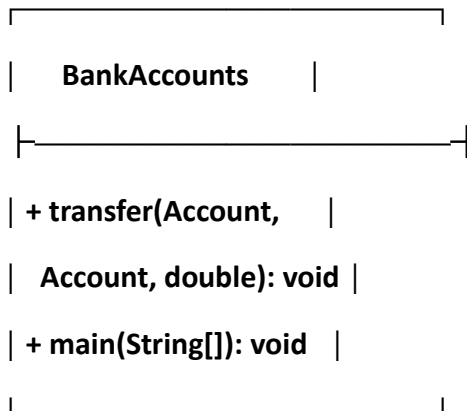
10.Display the updated savings account details.

11.Display the updated current account details.

12.Stop.

UML DIAGRAM:





CODE:

```
package javaprograms;
```

```
class Account {
```

```
    int accountNumber;
```

```
    double balance;
```

```
    String accountType;
```

```
    Account(int accountNumber, double balance, String accountType) {
```

```
        this.accountNumber = accountNumber;
```

```
        this.balance = balance;
```

```
        this.accountType = accountType;
```

```
    }
```

```
    void deposit(double amount) {
```

```
        if (amount > 0)
```

```
            balance += amount;
```

```
    }
```

```
    void withdraw(double amount) {
```

```

    if (amount > 0 && amount <= balance)
        balance -= amount;
    else
        System.out.println("Insufficient balance");
}

void calculateInterest() {
    System.out.println("No interest");
}

void display() {
    System.out.println("Account Number: " + accountNumber);
    System.out.println("Account Type: " + accountType);
    System.out.println("Balance: " + balance);
}
}

class SavingsAccount extends Account {
    private double interestRate;

    SavingsAccount(int no, double balance, double rate) {
        super(no, balance, "Savings Account");
        interestRate = rate;
    }

    @Override
    void calculateInterest() {
        double interest = balance * interestRate / 100;
    }
}

```

```
        balance += interest;

        System.out.println("Interest: " + interest);
    }
}
```

```
class CurrentAccount extends Account {

    private double overdraftLimit;

    CurrentAccount(int no, double balance, double limit) {
        super(no, balance, "Current Account");
        overdraftLimit = limit;
    }
}
```

```
@Override
void withdraw(double amount) {
    if (amount > 0 && amount <= balance + overdraftLimit)
        balance -= amount;
    else
        System.out.println("Exceeds overdraft limit");
}
}
```

```
public class BankAccounts {

    static void transfer(Account from, Account to, double amount) {
        if (amount > 0 && amount <= from.balance) {
            from.balance -= amount;
            to.balance += amount;
            System.out.println("Transfer successful");
        }
    }
}
```

```

    } else {
        System.out.println("Transfer failed");
    }
}

public static void main(String[] args) {
    SavingsAccount savings = new SavingsAccount(101, 10000, 5);
    CurrentAccount current = new CurrentAccount(102, 5000, 3000);

    savings.display();
    savings.deposit(2000);
    savings.calculateInterest();

    current.display();
    current.withdraw(7000);

    transfer(savings, current, 3000);

    savings.display();
    current.display();
}
}

```

SAMPLE OUTPUT:

```

/*Account Number: 101
Account Type: Savings Account
Balance: 10000.0
Interest: 600.0
Account Number: 102

```

Account Type: Current Account

Balance: 5000.0

Transfer successful

Account Number: 101

Account Type: Savings Account

Balance: 9600.0

Account Number: 102

Account Type: Current Account

Balance: 1000.0*/